PROGRAMACIÓN DINÁMICA



Antonio Miguel Almagro Valles
Javier Díaz Vilchez
Rodrigo láñez Huertas

Índice

Índice **2**
Objetivo **3**
Antecedentes **3**
Estudio de la Implementación **4**
CamionRecursive() 5
CamionRec() y CamionTable() 5
CamionTable() 6
CamionRec() 6
CamionDP() 6
CamionDP2() 7
CamionDP3() 7
CamionDPMP() 7
KnapsackGreedy() 8
Estudio Teórico **9**
CamionRecursive() 9
CamionRec() y CamionTable() 9
CamionDP() 9
CamionDP2() 10
CamionDP3() 10
CamionDPMP() 10
KnapsackGreedy() 11
Conclusión complejidad 11
Estudio experimental **12**
ANEXO **16**
BIBLIOGRAFÍA **37**

Objetivo

El objetivo de esta práctica es desarrollar una aplicación para el distribuidor de verduras Karrayon que le asesore en el llenado de su camión con aquellos objetos (verduras) que quepan en él y que le puedan proporcionar el mayor beneficio posible. Para ello, se utilizará el método algorítmico de programación dinámica.

La programación dinámica es un método para resolver problemas de optimización que exhiben las propiedades de superposición y subestructura óptima. La idea es simple, si tenemos un problema que se puede dividir en subproblemas más pequeños, entonces podemos resolver el problema original resolviendo los subproblemas de manera recursiva.

En el caso de Karrayon, el problema es cómo llenar su camión de manera que maximice el beneficio obtenido de la venta de las verduras, teniendo en cuenta el peso máximo autorizado (PMA) del camión. Este problema se puede resolver utilizando programación dinámica, donde cada subproblema representa el beneficio máximo que se puede obtener con un peso dado y un número determinado de verduras.

En esta práctica el líder ha sido Rodrigo Iáñez Huertas aunque todos los miembros del grupo han participado de manera activa en la realización de la misma. Rodrigo se ha encargado de supervisar cada una de las tareas que hemos realizado, que han sido:

- CamionRecursive() → Rodrigo
- CamionRec() → Javier
- CamionDP() → Rodrigo
- CamionDP2() → Rodrigo
- CamionDP3 → Antonio
- CamionDPMP() → Javier
- KnapSackGreedy() → Antonio
- Estudio de la implementación → Javier
- Estudio Teórico → Rodrigo
- Estudio experimental → Antonio

Estas han sido las tareas que se han asignado para cada uno, aunque siempre con la supervisión de los demás miembros del grupo. El trabajo extra que se ha realizado en la práctica pero que no se ve reflejado en la repartición de tareas se ha realizado, en su mayoría, de forma conjunta por los miembros del equipo.

Antecedentes

El motivo de la realización de esta práctica es ofrecer un estudio detallado, bien estructurado y ejemplificado de un problema que conseguimos resolver utilizando

programación dinámica. El problema que se nos plantea es llenar un camión con determinados objetos de manera que haya un mayor beneficio.

Durante el desarrollo de esta práctica hemos tenido en cuenta varias alternativas para que se cumpla nuestro objetivo. A continuación se encuentran las alternativas que se han contemplado:

- **Recursión simple**: En esta alternativa utilizamos una función recursiva para explorar todas las posibles opciones.
- **PD con Recursión**: Esta alternativa mejora la anterior ya que se utiliza una tabla bidimensional para almacenar los resultados de los subproblemas, evitando así redundancia en los cálculos.
- **PD iterativa no fraccionable**: En esta alternativa los objetos son indivisibles y los pesos números enteros exactos. PD iterativa con llenado de la matriz alternativo: En esta alternativa la tabla se llena de una manera más general, Para cada ítem y para cada capacidad, verifica si el ítem puede ser incluido. Si es así, actualiza la matriz de beneficios.
- **PD con array unidimensional:** En este enfoque se usa un array unidimensional en lugar de una tabla bidimensional para almacenar los resultados intermedios.
- **PD con objetos infinitos no fraccionables:** En este enfoque se nos plantea que ahora hay una cantidad infinita de objetos de tipos distintos.
- **Greedy objetos fraccionables.**

La solución que nosotros proponemos como las más óp.tima es _____, ya que esta alternativa nos garantiza la solución óptima con el mínimo de recursos

Estudio de la Implementación

Para esta práctica, vamos a proporcionar una solución al problema propuesto, el cual es el problema de la mochila o Knapsack. Para esto, se nos propone realizar una serie de algoritmos que utilicen un esquema de programación dinámica para resolver este problema. Como bien sabemos, los algoritmos de programación dinámica se basan en una estrategia de divide y vencerás o memorización, en la cual dividiremos el problema en diferentes subproblemas, de forma que posteriormente podamos llegar con la unión de estos subproblemas a la solución.

Comenzamos por comentar el desarrollo de este problema. Disponemos de una mochila con x capacidad y una serie de objetos con diferentes pesos y beneficios. La solución más óptima será aquella en la cual podamos encontrar el mayor beneficio llenando la mochila con los diferentes objetos sin rebasar la capacidad máxima de esta. Para ello hemos elaborado las siguientes clases (la documentación en JavaDoc se encuentra en las clases subidas todas en nuestro repositorio de github de la asignatura):

Objeto.java

Necesitaremos de una clase con la que podamos identificar a los objetos que dispondremos para cada problema. Para ello, elaboramos esta clase objeto, en la cual se define que cada objeto tiene un peso, beneficio, cantidad y nombre. Estos

atributos dispondrán de sus respectivos getters y setters, y la clase dispondrá de un método equals para comparar igualdad entre dos objetos en función de su nombre.

Knapsack.java

Esta es la clase principal donde se tratan las soluciones algorítmicas propuestas para el problema de la mochila. Contiene una lista de objetos, una variable P que indicará la capacidad máxima de la mochila y una matriz bidimensional de reales y un array de enteros que utilizaremos para los algoritmos de programación dinámica. A continuación, se detalla cada uno de los algoritmos propuestos:

CamionRecursive()

Este algoritmo resuelve el problema de la mochila utilizando un enfoque recursivo, con una estrategia de programación dinámica de “Divide y Vencerás”. El algoritmo funciona del siguiente modo, teniendo en cuenta que nos encontramos con 3 casos:

Si la capacidad de la mochila es 0 o no quedan objetos en la lista, significa que no quedan objetos por considerar o no se pueden agregar más en la mochila, por lo que el método devuelve 0 como beneficio. Este sería el caso base.

El segundo caso contempla si el peso del objeto actual es mayor que la capacidad restante de la mochila, lo que significa que no se puede incluir en la solución actual y lo descarta, llamando al método recursivamente con el siguiente objeto en la lista pasándole como n (número de objetos) = n - 1.

El último caso contempla si el peso del objeto es menor o igual que la capacidad restante de la mochila, lo que significa que es válido para ser agregado a la solución. Para este caso, se llama al método recursivamente 2 veces: la primera llamada incluyendo el objeto actual con n - 1 objetos y la capacidad de la mochila reducida con el peso de este objeto, que nos indica que se está incluyendo; y la segunda llamada sin incluir el objeto, para n - 1 objetos y la misma capacidad de la mochila. El beneficio mayor entre estas dos llamadas será la solución final.

CamionRec() y CamionTable()

Para este algoritmo, se propone resolver el problema de la mochila con una estrategia de programación dinámica de “Memorización”. El algoritmo utiliza una matriz bidimensional en la cual se almacenan los resultados de las llamadas recursivas previamente calculadas, lo que evita el recálculo que se realiza en el algoritmo anterior ya que almacenamos consecutivamente las soluciones en esta matriz. Pasamos a analizar ambos métodos:

CamionTable()

Éste método inicializa primeramente la matriz bidimensional con unos valores predeterminados de n + 1 filas y P + 1 columnas y posteriormente llama al método CamionRec() que es el que obtiene la solución óptima para alcanzar el mayor beneficio y el que almacena los objetos solución en la matriz. El método devolvería el beneficio máximo.

CamionRec()

En primer lugar, al igual que el algoritmo recursivo, verifica si la capacidad de la mochila es 0 o no quedan objetos, lo que significa que hemos concluído y devuelve 0 como beneficio (caso base).

En segundo caso, verifica si el objeto actual ya está almacenado en la matriz, devolviéndolo en caso afirmativo.

Cómo últimos casos, al igual que el algoritmo anterior, comprueba si el objeto tiene un peso mayor a la capacidad actual de la mochila, llamando al siguiente objeto en la lista y, como último caso en el que el objeto tenga un peso válido entre los límites del peso actual de la mochila, se calculan los beneficios incluyendo y excluyendo dicho objeto y se toma el máximo beneficio entre ellos, incluyendo finalmente dicho objeto en el conjunto de soluciones existentes en la matriz bidimensional.

CamionDP()

Para este algoritmo, se propone resolver el problema de la mochila con programación dinámica suponiendo que los objetos no se pueden fraccionar. Para ello, proponemos la siguiente implementación:

Primeramente se crea una matriz bidimensional con un número de filas = número de objetos y número de columnas = capacidad de la mochila + 1 y se inicializa a 0. Iteramos sobre la lista de objetos y para cada uno de ellos se calcula el beneficio máximo alcanzable para cada capacidad de la mochila desde 1 hasta P. Para cada capacidad de la mochila, se calcula el beneficio máximo considerando 2 casos, al igual que en los algoritmos anteriores:

. Incluyendo el objeto actual, de modo que se suma el beneficio del objeto actual al beneficio máximo alcanzable para la capacidad restante de la mochila.

. Excluyendo el objeto actual, de modo que el beneficio máximo es el beneficio máximo alcanzable para la misma capacidad de la mochila.

Comparando estos dos casos, se actualiza la matriz agregando el mayor beneficio de entre los 2 a esta. Al finalizar el método para todos los objetos, el último índice de la matriz contendrá el beneficio máximo alcanzado para dicha mochila con dichos objetos.

CamionDP2()

Para este algoritmo, se propone obtener la solución al problema de la mochila utilizando programación dinámica iterativa. Comienza al igual que el algoritmo anterior de CamionDP() inicializando la matriz bidimensional con número de objetos x capacidad de la mochila + 1. Como hemos dicho anteriormente, es un algoritmo iterativo, por lo que su desarrollo pasa por iterar mediante dos bucles anidados, en los cuales el primero itera sobre la lista de objetos y el segundo sobre las capacidades de la mochila desde 1 hasta su capacidad máxima.

En cada iteración del segundo bucle hasta llegar a la capacidad máxima de la mochila, se contempla el beneficio máximo que se puede obtener considerando el objeto actual y sin considerarlo solamente en caso de que su peso no exceda la capacidad máxima de la mochila (al igual que en los algoritmos anteriores, solo que esta vez de forma iterativa). Se almacenará en la tabla el beneficio máximo entre las dos opciones y, al finalizar los bucles anidados, se devuelve el beneficio máximo almacenado en la última posición de la matriz para P = su máxima capacidad.

CamionDP3()

Para este algoritmo, se propone resolver el problema de la mochila mediante programación dinámica utilizando solamente, en comparación con los anteriores, un array unidimensional.

El método comienza inicializando el array unidimensional con capacidad de P (capacidad máxima de la mochila) + 1. Contiene dos bucles anidados que iteran respectivamente sobre la lista de objetos a añadir y las capacidades de la mochila desde su capacidad máxima hasta 0, en comparación con el algoritmo anterior.

Para cada iteración del segundo bucle, se comprueba si el peso del objeto actual excede el peso de la mochila y, en caso negativo, se calcula el beneficio máximo que se puede obtener incluyendo el objeto actual y se actualiza el valor en el array comparando dicho valor con el valor máximo actual en dicha posición.

El método devuelve el beneficio máximo almacenado en la última posición del array, al igual que en los algoritmos anteriores.

CamionDPMP()

Para este algoritmo, se propone resolver el problema de la mochila para n diferentes tipos de objetos (es decir, de cada tipo de objeto podemos elegir la cantidad que se desee) con una estrategia de programación dinámica y utilizando un array unidimensional, al igual que el algoritmo anterior.

Para este caso en el que tenemos infinitos objetos de cada tipo, utilizamos otro enfoque distinto al iterar sobre las capacidades de la mochila y la lista de objetos. Para este caso, el primer bucle iteraría sobre las capacidades desde 0 hasta P (capacidad máxima) y el segundo bucle iterar sobre la lista de objetos. De este modo, comparando el máximo entre el beneficio obtenido incluyendo el objeto y excluyéndolo, al igual que hemos hecho en el algoritmo anterior, podríamos en cada iteración incluir cada objeto tantas veces como sea necesario en la mochila para maximizar el beneficio total. El método devuelve este beneficio máximo contenido en la última posición del array al igual que el algoritmo anterior de CamionDP3().

KnapsackGreedy()

Para este algoritmo, se propone encontrar una solución al problema de la mochila utilizando un enfoque voraz o greedy. Para este algoritmo, proponemos la siguiente implementación (recordamos de la práctica anterior que los algoritmos voraces son iterativos, con lo cual la solución a esta implementación será un algoritmo iterativo):

Las estructuras utilizadas son un array unidimensional de booleanos que almacenará qué objetos han sido usados previamente y otro array unidimensional de reales que contendrá la solución de los pesos empleados.

Se itera mediante dos bucles anidados, ambos sobre la lista de objetos. En cada iteración del bucle externo, se inicializa una variable “maxBeneficio” que se encargará de encontrar el beneficio máximo relativo y otra variable “pos” para rastrear la posición de este objeto. En cuanto al bucle interno, en cada iteración se comprueba si el objeto ha sido usado previamente mediante la comprobación en el array unidimensional de booleanos. Si no ha sido usado, se compara su beneficio con la variable previamente definida y si es mayor, se actualiza esta variable y se guarda su posición. Una vez iterado sobre todos los objetos, se verifica el peso del mayor objeto para comprobar si cabe dentro de la mochila y en caso afirmativo se incluye en el array de solución su peso, al igual que se marca como usado dentro del array de booleanos. Una vez completada la capacidad de la mochila o comprobado que no existen objetos por añadir, el método devolvería este array solución con los pesos de los objetos escogidos.

Estudio Teórico

CamionRecursive()

La complejidad temporal de este algoritmo es **O(nP)**, donde **n** es el número de elementos y **P** es la capacidad máxima del camión. Esto se debe a que cada combinación de **n** y **P** se calcula una sola vez y se almacena para su uso en cálculos futuros. Esto permite optimizar el tiempo de ejecución ya que evita realizar cálculos que ya se han hecho..

En cuanto a la complejidad espacial, en este algoritmo, no se necesita ninguna estructura de datos adicional para almacenar los resultados, lo que minimiza su complejidad. Sin embargo, si tenemos en cuenta la pila de llamadas del sistema, la cual en el peor de los casos es de tamaño **n**, la complejidad espacial sería **O(n)**.

CamionRec() y CamionTable()

En este caso, la complejidad temporal es **O(nP)**. Como en el caso anterior, n es el número total de elementos y P la capacidad total del camión. Esto se puede observar fácilmente ya que en el método **camionTable()** hay dos bucles for anidados que iteran sobre una tabla bidimensional llamada **tabla[][]**. El primer bucle for itera sobre el eje i, que representa el número de elementos, y el segundo sobre el eje j, que representa la capacidad del camión.

Esta tabla se inicializa con los valores **[n+1][P+1]**, lo que nos lleva a poder deducir también su complejidad espacial. En este caso, es también **O(nP)**.

Por tanto, tanto la complejidad espacial como la temporal son **O(nP)**. Esto significa que el tiempo de ejecución y el espacio de memoria aumentan linealmente conforme al número de elementos y la capacidad del camión. Esto es relativamente eficiente aunque para valores grandes no es un algoritmo óptimo.

CamionDP()

La complejidad temporal de este algoritmo es **O(nP)**, donde como siempre **n** es el número de elementos y **P** la capacidad total del camión. En este caso nos encontramos 3 bucles for, sin embargo, los dos últimos se ejecutan secuencialmente. El primero itera sobre los índices de los elementos, desde 1 hasta numItems. Luego se encuentran dos bucles secuenciales anidados al primero que iteran sobre **pesoActual**, que es **P**. En conclusión, tenemos un

bucle que se ejecuta **n** veces, y dentro tenemos otros dos bucles que se ejecutan **P** veces en total. Por lo que se puede deducir que la complejidad temporal del algoritmo es **O(nP)**.

Con respecto a la complejidad espacial, es la misma que el algoritmo anterior ya que se crea una matriz bidimensional llamada matrizBeneficios de tamaño **[n+1][P+1]**.

CamionDP2()

Este algoritmo tiene un estructura muy parecida a los anteriores, dos bucles for anidados, el primero itera sobre el número de elementos(**n**), y el segundo sobre la capacidad del camión (**P**). Por lo tanto al tratarse de dos bucles for anidados este algoritmo tiene una complejidad temporal **O(nP)**.

De la misma forma, en este caso, al crearnos un array bidimensional de tamaño **[n+1][P+1]**, la complejidad espacial del algoritmo es **O(nP)**. Por tanto, en términos de eficiencia, el algoritmo anterior y este, son bastante similares.

CamionDP3()

La complejidad temporal de este algoritmo es **O(nP)**. Es muy parecido a los anteriores, tiene dos bucles for anidados, el primero itera sobre el **numItems** y el segundo sobre **P**, siendo **numItems** el número de elementos y **P** la capacidad del camión.

Lo que cambia en este algoritmo con respecto a los anteriores, es que este trabaja con una matriz unidimensional de tamaño **[P+1]**. Por lo que se puede deducir que la complejidad espacial de este algoritmo es de **O(P)**.

En conclusión se puede decir que este último algoritmo es más eficiente que **camionDP(**) y **camionDP2()** ya que, a pesar de tener la misma complejidad temporal, este último utiliza menos espacio en memoria que los anteriores.

CamionDPMP()

La complejidad temporal de este método es **O(nP)** ya que, tiene dos bucles for anidados. El primero itera sobre el número de elementos y el segundo sobre la capacidad del camión. También es importante destacar que la complejidad de las operaciones **get** que hay en el segundo bucle son **O(1)**, por lo que no tienen valor a la hora de calcular su complejidad temporal.

Este algoritmo crea un array unidimensional de tamaño **[P+1]**, por lo que su complejidad espacial es **O(P)**.

KnapsackGreedy()

Este algoritmo, al igual que los anteriores, tiene una estructura de dos bucles for anidados, la única diferencia es que los dos bucles iteran sobre el número de elementos (**n**). El bucle interno busca el objeto con mayor beneficio unitario que aún no esté dentro del camión, y el bucle externo repite el proceso de búsqueda hasta que se llene el camión. Por lo que este algoritmo tiene una complejidad temporal **O(n^2)**.

La complejidad espacial del algoritmo es un tanto diferente, ya que en este caso nos creamos dos arrays unidimensionales: **usado** y **sol.** En el primero se insertan los elementos que han sido seleccionados, y el segundo almacena la solución final, ambos de tamaño **n**. Sabiendo esto se puede deducir que la complejidad espacial es **O(2n)**, que simplificando sería **O(n)**.

Conclusión complejidad

En esta práctica nos hemos centrado principalmente en el estudio de dos técnicas algorítmicas: programación dinámica y greedy.

La programación dinámica resuelve problemas dividiéndolos en subproblemas más pequeños y solucionando cada subproblema una sola vez, evitando así operaciones redundantes. Este tipo de algoritmos son muy eficientes para resolver problemas de optimización con una estructura de problemas superpuestos. También es destacable que siempre encuentra la solución óptima al problema. Por otro lado, este tipo de algoritmos suelen usar una gran cantidad de memoria, especialmente si hacemos uso de matrices bidimensionales para almacenar los resultados auxiliares, además de ser relativamente difíciles de implementar.

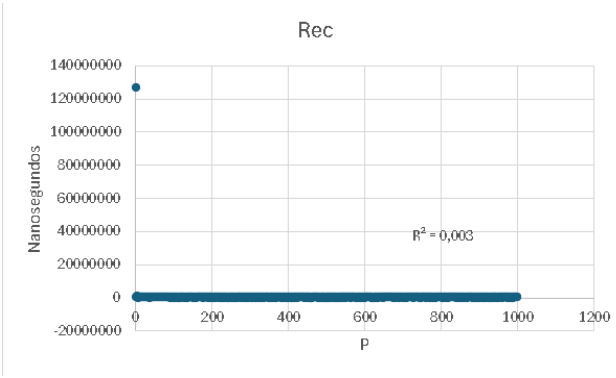
Los algoritmos greedy funcionan paso a paso, es decir, toman la decisión óptima del momento con la intención de buscar la solución global óptima, aunque no la garantizan, ya que no vuelven sobre decisiones pasadas. La principal ventaja de este tipo de algoritmos es que estos algoritmos son generalmente más fáciles de implementar ya que siguen una estructura lógica al pensamiento humano. También suelen ser más óptimos en términos de complejidad. Sin embargo, la principal desventaja de un algoritmo greedy es que no te garantiza la solución óptima. En su lugar, encuentra la óptima en cada paso, lo que no te garantiza la óptima final.

En conclusión, si el problema a resolver tiene una estructura de subproblemas superpuestos o tenemos la necesidad de encontrar la solución óptima final, es recomendable usar la programación dinámica.

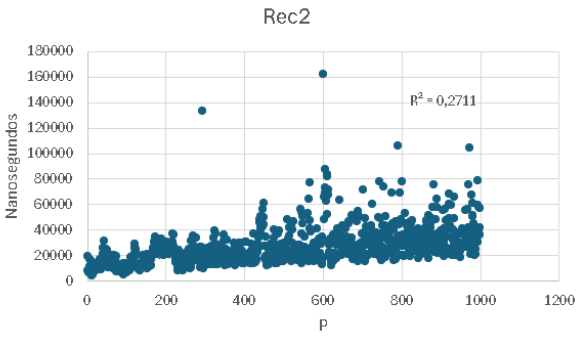
Estudio experimental

Para N constante y P variable

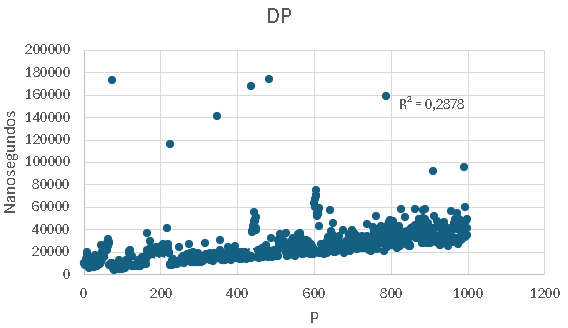
Recursivo



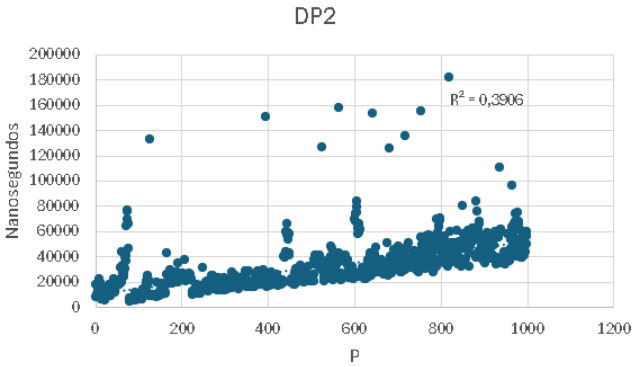
Recursivo2



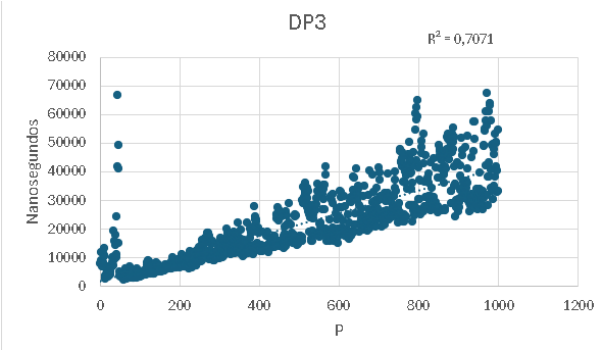
DP



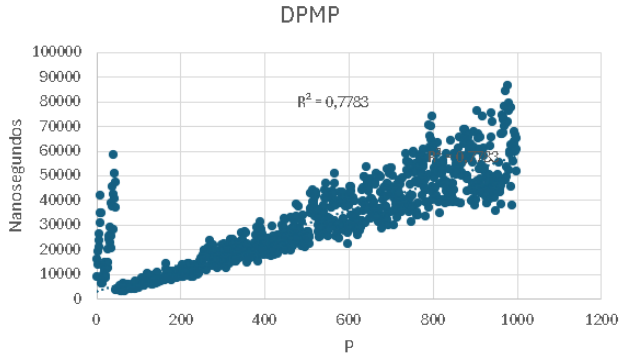
DP2



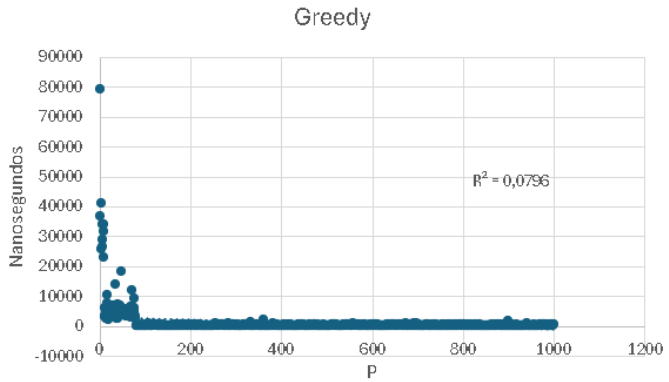
DP3



DPMP

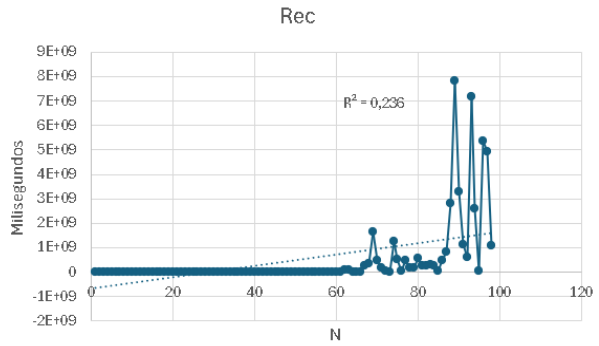


GREEDY

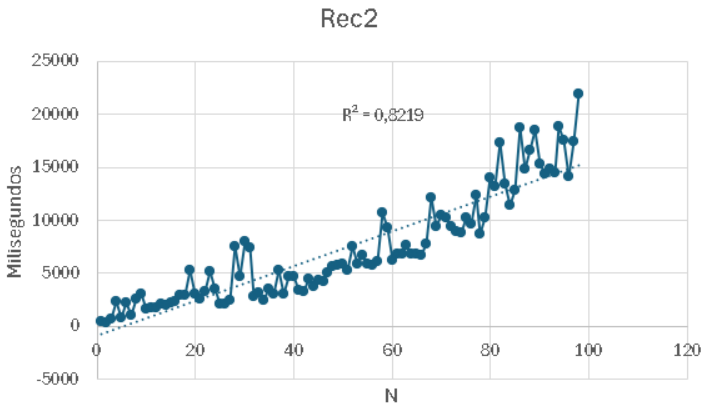


Para N variable y P constante

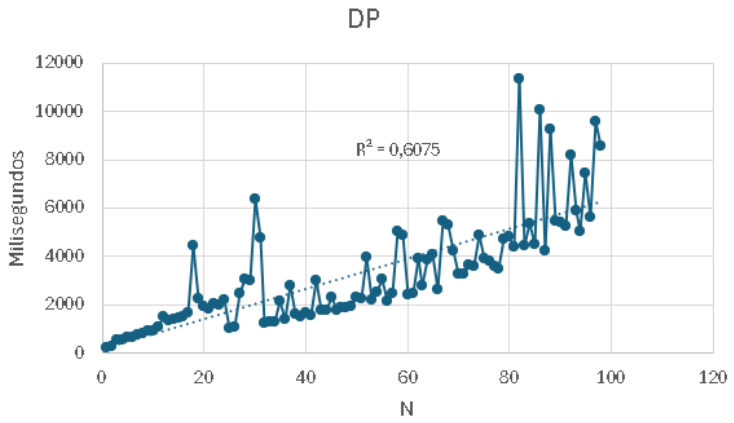
Recursivo



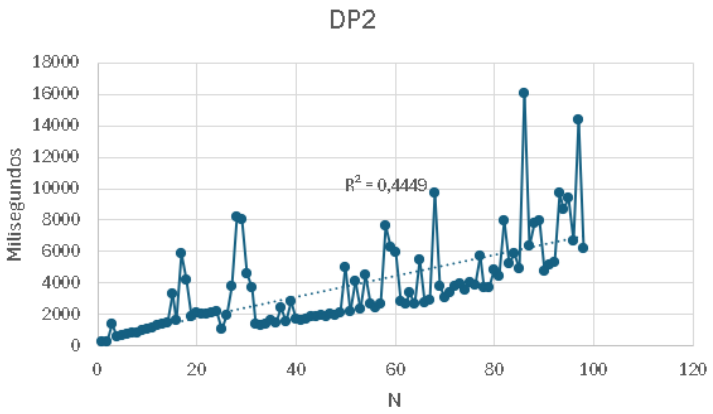
Recursivo2



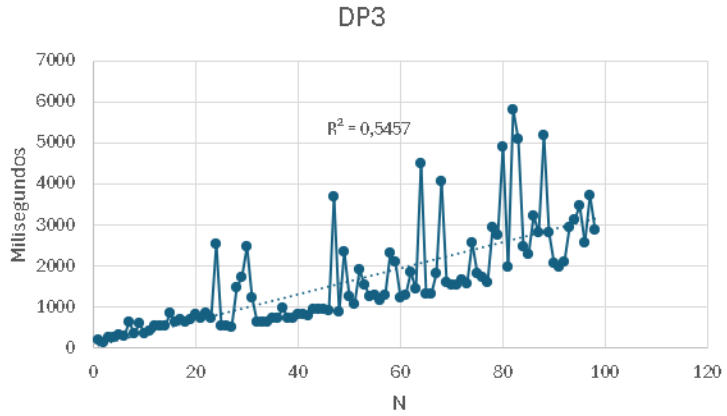
DP



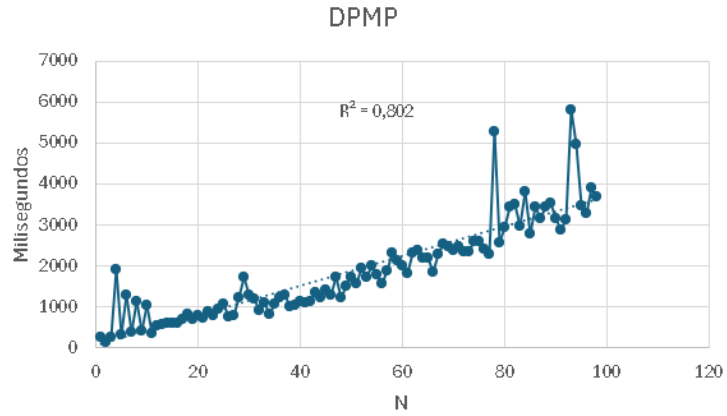
DP2



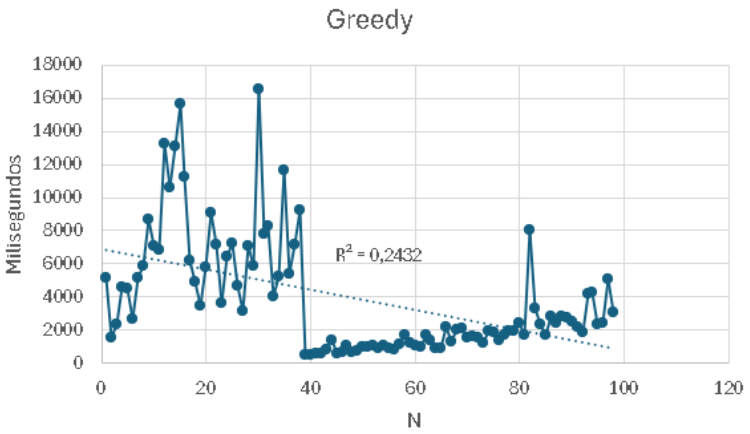
DP3



DPMP



GREEDY



Como se puede observar arriba, hemos generado resultados variando N o P, siendo N el número de objetos y P la capacidad de la mochila, el resultado se nos expresa en nanosegundos. Para N constante hemos obtenido que el peor resultado ha sido el recursivo y el mejor el DPMP, y para N variable ha sido el recursivo 2, el mejor, y el peor ha sido el recursivo. Como R^2 ha sido por debajo de 0,90 no podemos decir que la toma de datos ha sido tomada correctamente, por lo que el estudio experimental podría cambiar con respecto a lo demostrado en el estudio teórico.

Anexo

Cuestiones

1. **Implementar en Java un método recursivo que, verificando la recurrencia anterior, devuelva el máximo beneficio que podría transportar el camión si éste tiene un PMA de P. Tenemos un array p con los pesos de los objetos, y otro array b con sus beneficios. La cabecera de la función a implementar puede ser:**
   double camionRecursive(int n, int P, int p[], double b[])

   Para poder comprobar que el funcionamiento del método camionRecursive se ha generado una mochila aleatoria:

```
Generando mochila aleatoria...
Introduzca el número de objetos:
10
Introduzca la capacidad máxima de la mochila:
30
MOCHILA (30)
Objeto 1        p=25,0        b=93,0
Objeto 2        p=19,0        b=39,0
Objeto 3        p=5,0         b=53,0
Objeto 4        p=5,0         b=64,0
Objeto 5        p=29,0        b=60,0
Objeto 6        p=9,0         b=78,0
Objeto 7        p=19,0        b=70,0
Objeto 8        p=13,0        b=19,0
Objeto 9        p=17,0        b=79,0
Objeto 10       p=4,0         b=78,0

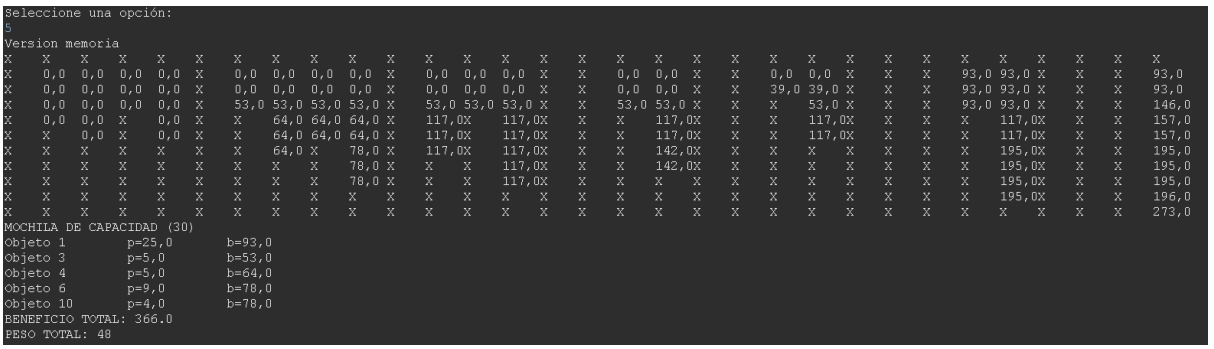
Mochila generada correctamente
```

Después hemos ejecutado dicho método, obteniendo que el beneficio total es de 273, como se puede comprobar abajo.

```
Seleccione una opción:
4
Version recursiva
BENEFICIO TOTAL: 273.0
```

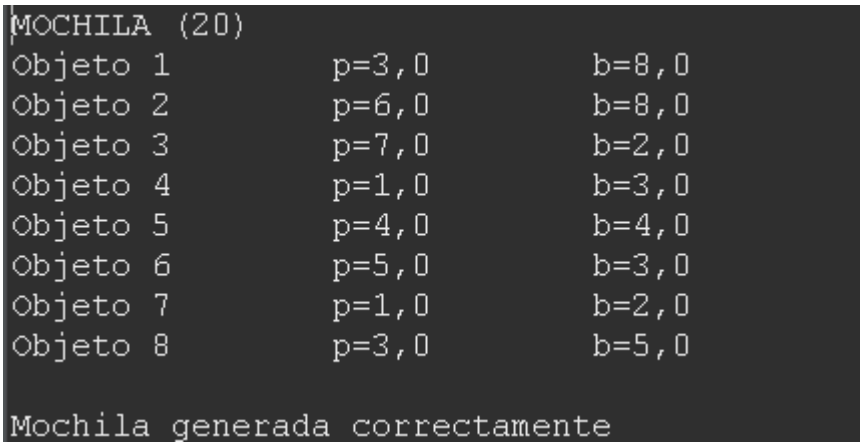
2. **Implementar en Java un método que evite el recálculo que se realiza en el caso anterior utilizando únicamente la recursividad. Para ello, deberá utilizar una tabla (matriz bidimensional) con n + 1 filas y P + 1 columnas en la que se irán almacenando los valores para ser utilizados posteriormente (muestre el valor final que tendría dicha tabla e interprételo) y no necesitar el recálculo que se hace con la recursividad. Para ello se deben implementar dos funciones (camionTable, que llama internamente a camionRec), cuyas cabeceras pueden ser:**
   double camionRec(int n, int P, int p[], double b[], double[][] table)
   double camionTable(int n, int P, int p[], double b[]))

   En este ejercicio hemos utilizado la mochila generada para el ejercicio 1, y hemos ejecutado el método camionRec, donde se puede ver que usa una matriz bidimensional que hemos llamado camionTable.



3. **Suponiendo que los objetos (verduras) no se pueden fraccionar (romper) y que los pesos son exactos (número naturales positivos), determinar qué objetos debe elegir Karryon para maximizar el valor total de lo que pueda llevarse en su camión, sabiendo que los precios (beneficios) de los objetos que ofrece EDAMarket-Almeria son números reales positivos. Implementar en Java la solución al problema, según el algoritmo visto en clase (knapSackDP) en su versión iterativa (utilizando Programación Dinámica) y exponer el resultado en función de array Sol ($Sol[i] \in \{0, 1\}$) y en función de lo que proporciona la función test, que nos proporciona realmente los objetos a considerar según la solución Sol. Probar su correcto funcionamiento para el caso: P = 20, p = {3, 6, 7, 1, 4, 5, 1, 3}, b = {8.0, 8.0, 2.0, 3.0, 4.0, 3.0, 2.0, 5.0}. Para ello se deben implementar dos funciones (camionDP y test), cuyas cabeceras pueden ser:**
   double camionDP(int P, int p[], double b[])
   void test(int j, int c, int p[], double b[], double camion[][])

   Para realizar correctamente este ejercicio, hemos tenido que crear una mochila de manera manual, donde hemos metido los parámetros que se nos han pedido en el enunciado.



   Con la mochila ya generada, pasamos a ejecutar el método camionDP, dándonos, con una mochila de capacidad 20, un beneficio total de 30 y un peso total de 18.

```
Version DP
0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0
0,0  0,0  0,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0
0,0  0,0  0,0  8,0  8,0  8,0  8,0  8,0  8,0  16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0
0,0  0,0  0,0  8,0  8,0  8,0  8,0  8,0  8,0  16,0 16,0 16,0 16,0 16,0 16,0 16,0 18,0 18,0 18,0 18,0 18,0 18,0
0,0  3,0  3,0  8,0  11,0 11,0 11,0 11,0 11,0 16,0 19,0 19,0 19,0 19,0 19,0 19,0 19,0 21,0 21,0 21,0 21,0 21,0
0,0  3,0  3,0  8,0  11,0 11,0 11,0 12,0 15,0 16,0 19,0 19,0 19,0 20,0 23,0 23,0 23,0 23,0 23,0 23,0 23,0 23,0
0,0  3,0  3,0  8,0  11,0 11,0 11,0 12,0 15,0 16,0 19,0 19,0 19,0 20,0 23,0 23,0 23,0 23,0 23,0 26,0 26,0 26,0
0,0  3,0  5,0  8,0  11,0 13,0 13,0 13,0 15,0 17,0 19,0 21,0 21,0 21,0 23,0 25,0 25,0 25,0 25,0 26,0 28,0 28,0
0,0  3,0  5,0  8,0  11,0 13,0 13,0 16,0 18,0 18,0 19,0 21,0 22,0 24,0 26,0 26,0 26,0 28,0 30,0 30,0 30,0 30,0
MOCHILA DE CAPACIDAD (20)
Objeto 1        p=3,0        b=8,0
Objeto 2        p=6,0        b=8,0
Objeto 4        p=1,0        b=3,0
Objeto 5        p=4,0        b=4,0
Objeto 7        p=1,0        b=2,0
Objeto 8        p=3,0        b=5,0
BENEFICIO TOTAL: 30.0
PESO TOTAL: 18
```

4. **Si mostramos también los valores que tiene la tabla o matriz bidimensional camión (variable local al método) del algoritmo del apartado 3, ¿Se puede deducir alguna relación con los valores de la matriz bidimensional table del caso 2?. Exponerlo todo de forma razonada.**

   Si existe una relación entre los valores de ambas tablas aunque la tabla sea diferente ya que ambas tablas representan lo mismo: el beneficio máximo que se puede obtener para cada subconjunto de objetos y cada capacidad del camión. En ambos casos, la fila i de la tabla representa el número de elementos, y la fila j, la capacidad del camión. La única diferencia es la forma en la que se llenan. En el caso de la matriz del caso 2, esta se llena de una manera más general. Para cada objeto y para cada capacidad, simplemente verifica si el objeto puede ser incluido (es decir, si su peso es menor o igual a la capacidad actual). En la matriz del apartado 3, esta se llena de otra forma menos general. Para cada objeto, primero se llenan las posiciones de la matriz cuyo peso es menor que el del objeto actual(en las cuales el objeto no puede ser incluido). Luego, para las capacidades que son mayores al peso del objeto con el que estamos trabajando.

   Aunque la forma de llenar la tabla es diferente, el resultado para el mismo conjunto de objetos y la misma capacidad debería de ser el mismo.

5. **Implementar en Java, la otra forma de resolver el problema (otra forma de llenar la matriz B de forma iterativa, en este caso será la matriz camion) vista en clase (Algoritmo 6.4 en la transparencia 93), y obtener la 'salida de la función test' sin necesidad de utilizar dicha función recursiva (fragmento de código iterativo). Probarlo también para el mismo caso del apartado 3 (P, p[], b[]). Para ello se deben implementar una única función (camionDP2), cuya cabecera puede ser:**
   double camionDP2(int P, int p[], double b[])

```
Version memoria

-    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -
-    0,0  0,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0
-    0,0  0,0  8,0  8,0  8,0  8,0  8,0  8,0  16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0
-    -    -    -    -    -    8,0  8,0  8,0  16,0 16,0 16,0 16,0 16,0 16,0 16,0 18,0 18,0 18,0 18,0 18,0
-    -    -    -    -    -    -    11,0 11,0 -    19,0 19,0 19,0 19,0 19,0 19,0 19,0 21,0 -    21,0 21,0
-    -    -    -    -    -    -    -    -    -    -    19,0 19,0 -    23,0 23,0 23,0 23,0 -    23,0 23,0
-    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    23,0 23,0 -    26,0 26,0
-    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    25,0 -    -    28,0
-    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    -    30,0
La capacidad es:  (20)
1 | Beneficio: 8.0 | Peso: 3.0
2 | Beneficio: 8.0 | Peso: 6.0
4 | Beneficio: 3.0 | Peso: 1.0
5 | Beneficio: 4.0 | Peso: 4.0
7 | Beneficio: 2.0 | Peso: 1.0
8 | Beneficio: 5.0 | Peso: 3.0
El beneficio total del camión es: 30.0€
El peso total del camión es: 18kg
```

```
Version DP
0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0
0,0  0,0  0,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0  8,0
0,0  0,0  0,0  8,0  8,0  8,0  8,0  8,0  8,0  16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0 16,0
0,0  0,0  0,0  8,0  8,0  8,0  8,0  8,0  8,0  16,0 16,0 16,0 16,0 16,0 16,0 16,0 18,0 18,0 18,0 18,0
0,0  3,0  3,0  8,0  11,0 11,0 11,0 11,0 11,0 16,0 19,0 19,0 19,0 19,0 19,0 19,0 21,0 21,0 21,0 21,0
0,0  3,0  3,0  8,0  11,0 11,0 11,0 12,0 15,0 16,0 19,0 19,0 19,0 20,0 23,0 23,0 23,0 23,0 23,0 23,0
0,0  3,0  3,0  8,0  11,0 11,0 11,0 12,0 15,0 16,0 19,0 19,0 19,0 20,0 23,0 23,0 23,0 23,0 23,0 26,0
0,0  3,0  5,0  8,0  11,0 13,0 13,0 13,0 15,0 17,0 19,0 21,0 21,0 21,0 23,0 25,0 25,0 25,0 25,0 26,0
0,0  3,0  5,0  8,0  11,0 13,0 13,0 16,0 18,0 18,0 19,0 21,0 22,0 24,0 26,0 26,0 26,0 28,0 30,0 30,0
MOCHILA DE CAPACIDAD (20)
Objeto 1        p=3,0        b=8,0
Objeto 2        p=6,0        b=8,0
Objeto 4        p=1,0        b=3,0
Objeto 5        p=4,0        b=4,0
Objeto 7        p=1,0        b=2,0
Objeto 8        p=3,0        b=5,0
BENEFICIO TOTAL: 30.0
PESO TOTAL: 18
```

Ambos obtienen un beneficio total de 30 euros y un peso total de 18 kilos, podemos encontrar las diferencias en las tablas que se generan.

Versión memoria:

- Se utiliza un array unidimensional que almacena los máximos beneficios para cada peso posible. Implicando que la tabla de beneficios se represente de manera más compacta y eficiente en términos de espacio
- Se actualiza el valor en “array[j]” al máximo entre el valor actual y el valor obtenido al agregar el objeto actual al camión

Versión DP:

- Se utiliza una matriz bidimensional para almacenar los máximos beneficios alcanzados para cada combinación de objeto y peso posible. Esto implica que la tabla de beneficios se representa de manera más explícita y fácilmente interpretable.
- Se calcula y actualiza el valor tomando en cuenta se se incluye o no el objeto actual en el camión.

6. **¿Sería posible encontrar el mayor beneficio o beneficio total que se obtiene al resolver nuestro problema con programación dinámica, pero utilizando únicamente un array unidimensional (camion) como estructura de datos, en**

lugar de utilizar una tabla o array bidimensional como hemos visto en los anteriores casos?. De ser posible, implementarlo en Java con la siguiente cabecera y explicar su funcionamiento de forma detallada:
double camionDP3(int P, int p[], double b[])
https://www.enjoyalgorithms.com/blog/zero-one-knapsack-problem
¿Sería sencillo en este caso encontrar el array Sol ($Sol[i] \in \{0, 1\}$)?. Justifique su respuesta.

Si sería posible para demostrarlo hemos generado lo siguiente:

1. Una función camionDP3(int P, int p[], double b[]) toma tres argumentos:
   P: el peso máximo admitido (PMA) del camión.
   p[]: un array que contiene los pesos de los objetos.
   b[]: un array que contiene los beneficios de los objetos.

2. Hemos declarado una variable n que representa la cantidad de objetos disponibles (la longitud de los arrays p[] y b[]).

3. Creamos un array unidimensional llamado array con una longitud de P + 1. Este array se utilizará para almacenar los beneficios máximos alcanzados para cada peso posible desde 0 hasta P.

4. Inicializamos un 0 bucle for para iterar sobre cada objeto disponible (i representa el índice del objeto).

5. Dentro del bucle for, se inicializa otro bucle for que itera desde P hasta 0. Este bucle itera desde el peso máximo admitido (PMA) hasta 0, lo que representa la capacidad disponible del camión.

6. Dentro del bucle interior, se comprueba si el peso del objeto actual (p[i-1]) es menor o igual al peso actual (j). Si es así, significa que el objeto puede ser incluido en el camión sin exceder el límite de peso.

7. Si se cumple la condición anterior, se actualiza el valor en array[j] al máximo entre el valor actual en array[j] y el valor obtenido al agregar el objeto actual al camión (array[j - p[i-1]] + b[i-1]). Esto garantiza que se mantenga el máximo beneficio alcanzado hasta el momento para el peso actual.

8. Una vez completadas todas las iteraciones, el valor en array[P] contendrá el máximo beneficio que se puede obtener con el peso máximo admitido (PMA).

9. Finalmente, se devuelve el valor en array[P], que representa el máximo beneficio posible que se puede obtener al llenar el camión con el PMA especificado.

En este caso no sería tan sencillo encontrar el array de solución, ya que se requiere modificaciones adicionales en la implementación y se aumentaría la complejidad del algoritmo.

7. **Este apartado es obligatorio para los grupos de 3 miembros y optativo para los grupos de 2. Suponiendo que ahora hay una cantidad inagotable de objetos de tipos (clases) distintos. Es decir, en vez tener n objetos distintos, disponemos ahora de n tipos de objetos distintos. Cada tipo de objeto es indivisible (no se puede romper), tiene un cierto peso (natural positivo) y un cierto beneficio (real positivo). Lo que se pretende es encontrar qué cantidad de objetos de cada tipo se debe coger para maximizar el valor total de lo que puede llevarse en el camión con PMA P.**

   a) Indicar la expresión de recurrencia para este problema de una forma similar a como se ha expuesto para camion(i, j), y exponer la similitud con la recurrencia que tenemos para el problema del cambio de monedas, que nos puede servir para su implementación.

La función de recurrencia para este problema es:

$$camion(i,j)\begin{cases} 0 & si\ i=0\ o\ j=0 \\ camion(i-1,j) & si\ peso[i]>j \\ \max\big(camion(i-1,j), beneficio[i]+camion(i,j-peso[i])\big) & si\ peso[i]<j \end{cases}$$

La similitud de esta con la recurrencia del problema del cambio de monedas es que en ambos casos, se está tratando de maximizar una cantidad. En el caso del camión, el beneficio, y en el caso del cambio de monedas el número de ellas. Teniendo en cuenta siempre una restricción que nos pone el propio problema, ya sea o bien la capacidad del camión o la cantidad a cambiar. La principal diferencia entre estos dos problemas es que en el problema del camión lo que pretendemos es maximizar el beneficio, mientras que en el de las monedas pretendemos minimizar el número de monedas a cambiar. Sin embargo estos dos problemas son muy similares, por lo que la forma de resolverlos también lo será.

b) Implementar en Java el algoritmo y probarlo para el caso del apartado 3 (P, p[], b[]). Para ello se debe implementar una única función (camionDPMP), cuya cabecera puede ser:
double camionDPMP(int P, int p[], double b[])

```
Version DPMP
MOCHILA DE CAPACIDAD (20)
Objeto 4        p=1,0        b=3,0        q=20
BENEFICIO TOTAL: 60.0
PESO TOTAL: 20
```

c) Intente implementar en Java la forma de obtener el array Sol ($Sol[i] \in \{0, 1\}$) para este caso. Y justifíquelo todo detalladamente.

Para obtener el arraySol tenemos el método “mejorSolucion()”, indicando la cantidad de objetos de cada tipo que se deben seleccionar para maximizar el valor total que se puede transportar en el camión con la capacidad máxima especificada(PMA P), para ello, arraySol se genera iterando sobre los objetos disponibles y seleccionando aquellos que maximizan el valor total sin exceder la capacidad del camión, se escoge el objeto que tiene mayor valor total por unidad de peso añadido, después, se añade la cantidad máxima posible de este objeto en el camión y se repite el proceso con los objetos restantes, hasta que no haya capacidad suficiente en el camión o no queden más objeto por añadir.

8. **Suponiendo que los objetos no se pueden fraccionar, pero los pesos pueden no ser exactos (los pesos pueden ser números reales positivos). Indicar razonadamente qué tendría de especial este caso con respecto a los apartados vistos anteriormente, por ejemplo, para el apartado 3.**

   Con la implementación realizada en el apartado 3, no nos valdría para este caso, ya que estamos trabajando con una tabla bidimensional de enteros, sería necesario un array de dobles. Por tanto, la complejidad aumentaría a nuestro problema y se requeriría ajustes significativos en la implementación del algoritmo.

9. **Finalmente, imaginemos el caso de que los objetos se pueden fraccionar, ¿sería sencillo encontrar una solución con programación dinámica para determinar qué objetos elegir para maximizar el valor total de lo que puede llevarse en el camión?. ¿Sería más sencillo realizarlo con un algoritmo greedy (voraz)?. De ser así, implementarlo y probarlo para varios casos. Además, exponer las principales diferencias (en forma de tabla) entre los dos esquemas algorítmicos (programación dinámica y greedy) para este problema concreto.**

   Cuando los objetos se pueden fraccionar la elección entre qué algoritmo utilizar depende de varios factores.

   Si elegimos usar programación dinámica la implementación puede ser más tediosa y compleja. Esta requiere un conocimiento profundo de este tipo de algoritmos además de que requiere más memoria para ser capaz de almacenar las soluciones a cada uno de los subproblemas resueltos. Lo bueno es que este algoritmo nos garantiza al 100% el resultado óptimo general.

   Si se elige la opción greedy, hemos de saber que su implementación va a ser más sencilla. Además de ser una implementación más eficiente en términos de espacio y de tiempo. Sin embargo, este método no nos garantiza el resultado óptimo final.

   En nuestro caso, la opción greedy es la adecuada, ya que al tratarse de objetos fraccionados, obtenemos el resultado óptimo. Tomar el objeto con la mayor relación valor/peso disponible en cada paso lleva a una solución óptima global. Esto se debe

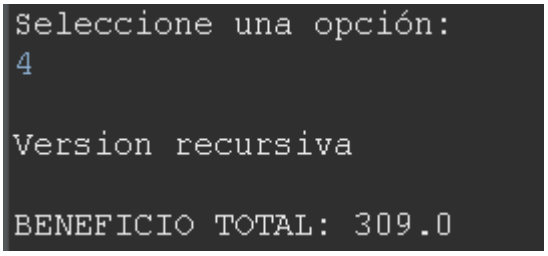
a que podemos fraccionar los objetos, lo que nos garantiza rellenar el camión en su totalidad.

	Programación dinámica	Greedy
Ventajas	- Al almacenar soluciones a subproblemas previamente calculados, acelera el proceso. - Convierte problemas complejos, en varios más sencillos.	- Son rápidos y fáciles de implementar. - Son más eficientes que otros métodos algoritmos. - Son ideales para problemas en los que no es necesario obtener la solución óptima.
Desventajas	- Es más lento, complejo y tedioso para implementar. - Requiere un buen conocimiento de este tipo de estructuras. - Requiere más memoria ya que almacena los valores de los resultados previamente calculados.	- No siempre garantizan obtener la solución óptima. - No tienen posibilidad de volver atrás a cambiar una decisión que se tomó.

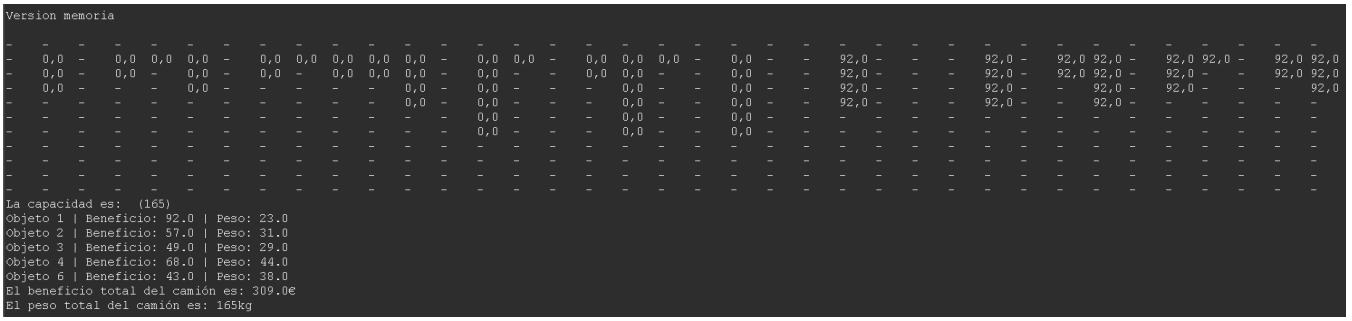
Carga de archivos

P01

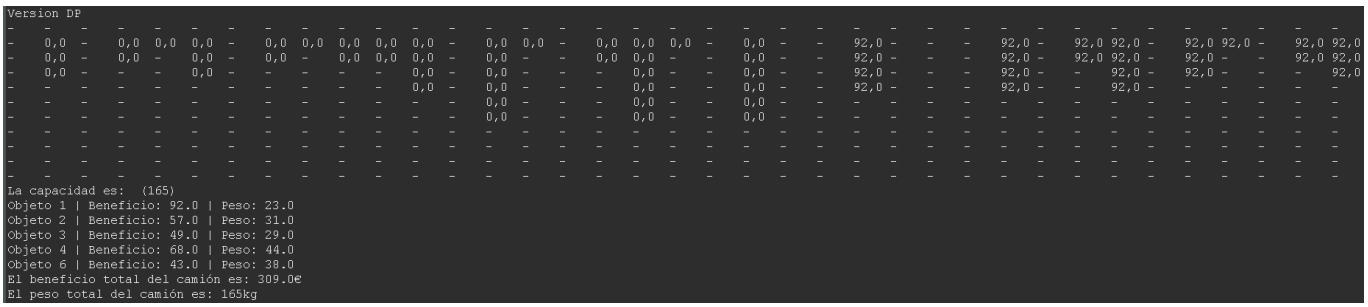
CamionRecursive



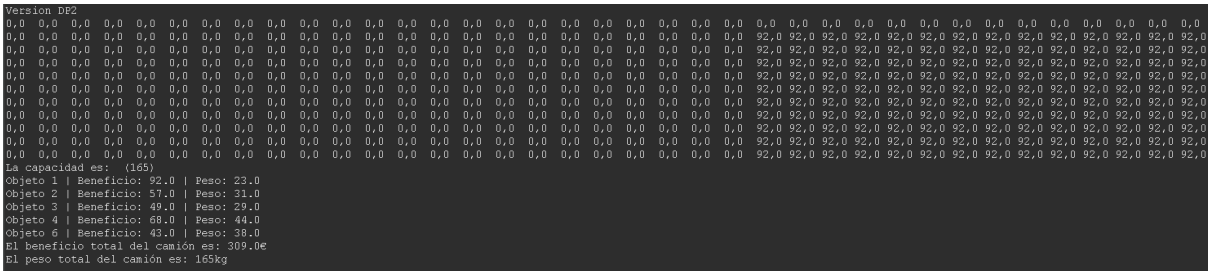
CamionTable



CamionDP



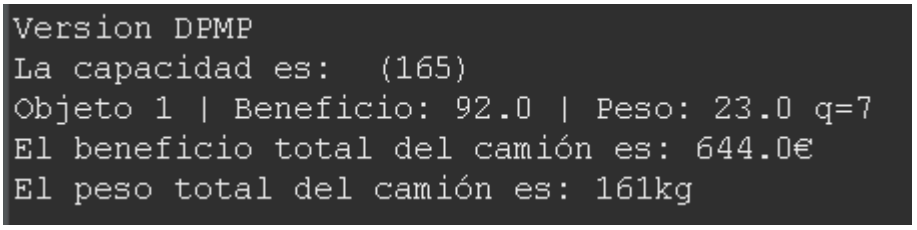
CamionDP2



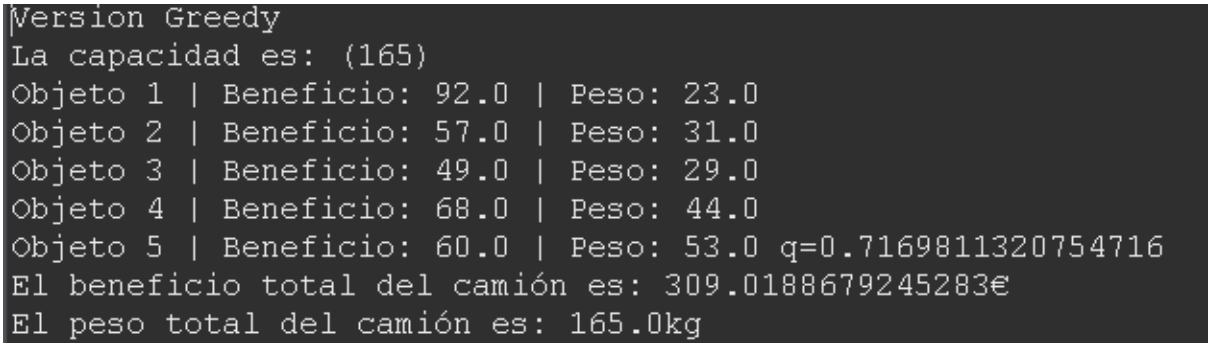
CamionDP3



CamionDPMP



Greedy



P02

CamionRecursive

```
Cargando archivo "p02_c.txt"...
Archivo cargado correctamente
==========================================
                  MENU
==========================================
1.  Cargar archivo
2.  Generar mochila aleatoria
3.  Crear mochila manual
4.  Ejecutar Camion Recursive
5.  Ejecutar Camion Table
6.  Ejecutar Camion DP
7.  Ejecutar Camion DP2
8.  Ejecutar Camion DP3
9.  Ejecutar Camion DPMD
10. Ejecutar Camion Greedy
11. Ejecutar pruebas de rendimiento
12. Salir
==========================================
Seleccione una opción:
4
Version recursiva
BENEFICIO TOTAL: 51.0
```

CamionTable

```
Seleccione una opción:
5
Version memoria
X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X
X    X    0,0  X    X    X    0,0  0,0  0,0  0,0  0,0  0,0  X    X    X    24,0 X    24,0 24,0 24,0 X    X    X    X    X    X    24,0
X    X    X    X    X    X    0,0  13,0 X    13,0 X    X    X    X    X    24,0 X    24,0 24,0 X    X    X    X    X    X    X    37,0
X    X    X    X    X    X    X    X    X    13,0 X    X    X    X    X    X    X    24,0 36,0 X    X    X    X    X    X    X    47,0
X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    28,0 X    X    X    X    X    X    X    X    51,0
X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    X    51,0
MOCHILA DE CAPACIDAD (26)
Objeto 2        p=7,0        b=13,0
Objeto 3        p=11,0       b=23,0
Objeto 4        p=8,0        b=15,0
BENEFICIO TOTAL: 51.0
PESO TOTAL: 26
```

CamionDP

```
Seleccione una opción:
6
Version DP
0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0
0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0
0,0  0,0  0,0  0,0  0,0  0,0  0,0  13,0 13,0 13,0 13,0 13,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 37,0 37,0 37,0 37,0 37,0 37,0 37,0 37,0 37,0
0,0  0,0  0,0  0,0  0,0  0,0  0,0  13,0 13,0 13,0 13,0 23,0 24,0 24,0 24,0 24,0 24,0 24,0 36,0 37,0 37,0 37,0 37,0 47,0 47,0 47,0 47,0
0,0  0,0  0,0  0,0  0,0  0,0  0,0  13,0 15,0 15,0 15,0 23,0 24,0 24,0 24,0 28,0 28,0 28,0 36,0 38,0 39,0 39,0 39,0 47,0 47,0 47,0 51,0
0,0  0,0  0,0  0,0  0,0  0,0  0,0  13,0 15,0 16,0 16,0 23,0 24,0 24,0 24,0 28,0 29,0 31,0 36,0 38,0 39,0 40,0 40,0 47,0 47,0 47,0 51,0
MOCHILA DE CAPACIDAD (26)
Objeto 2        p=7,0        b=13,0
Objeto 3        p=11,0       b=23,0
Objeto 4        p=8,0        b=15,0
BENEFICIO TOTAL: 51.0
PESO TOTAL: 26
```

CamionDP2

```
Seleccione una opción:
7
Version DP
0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0
0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  0,0  24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0
0,0  0,0  0,0  0,0  0,0  0,0  0,0  13,0 13,0 13,0 13,0 13,0 24,0 24,0 24,0 24,0 24,0 24,0 24,0 37,0 37,0 37,0 37,0 37,0 37,0 37,0 37,0
0,0  0,0  0,0  0,0  0,0  0,0  0,0  13,0 13,0 13,0 13,0 23,0 24,0 24,0 24,0 24,0 24,0 24,0 36,0 37,0 37,0 37,0 37,0 47,0 47,0 47,0 47,0
0,0  0,0  0,0  0,0  0,0  0,0  0,0  13,0 15,0 15,0 15,0 23,0 24,0 24,0 24,0 28,0 28,0 28,0 36,0 38,0 39,0 39,0 39,0 47,0 47,0 47,0 51,0
0,0  0,0  0,0  0,0  0,0  0,0  0,0  13,0 15,0 16,0 16,0 23,0 24,0 24,0 24,0 28,0 29,0 31,0 36,0 38,0 39,0 40,0 40,0 47,0 47,0 47,0 51,0
MOCHILA DE CAPACIDAD (26)
Objeto 2        p=7,0        b=13,0
Objeto 3        p=11,0       b=23,0
Objeto 4        p=8,0        b=15,0
BENEFICIO TOTAL: 51.0
PESO TOTAL: 26
```

CamionDP3

```
Seleccione una opción:
8
Version DP3
51.0
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 13.0, 15.0, 16.0, 16.0, 23.0, 24.0, 24.0, 24.0, 28.0, 29.0, 31.0, 36.0, 38.0, 39.0, 40.0, 40.0, 47.0, 47.0, 47.0,
BENEFICIO TOTAL: 51.0
```

CamionDPMP

```
Seleccione una opción:
9
Version DPMP
MOCHILA DE CAPACIDAD (26)
Objeto 2        p=7,0        b=13,0
Objeto 3        p=11,0       b=23,0
Objeto 4        p=8,0        b=15,0
BENEFICIO TOTAL: 51.0
PESO TOTAL: 26
```

Greedy

```
Seleccione una opción:
10
Version Greedy
MOCHILA DE CAPACIDAD (26)
Objeto 1        p=12,0       b=24,0
Objeto 3        p=11,0       b=23,0
Objeto 4        p=8,0        b=15,0       q=0.375
BENEFICIO TOTAL: 52.625
PESO TOTAL: 26.0
```

P03

CamionRecursive

```
Cargando archivo "p03_c.txt"...
Archivo cargado correctamente
==========================================
                  MENU
==========================================
1.  Cargar archivo
2.  Generar mochila aleatoria
3.  Crear mochila manual
4.  Ejecutar Camion Recursive
5.  Ejecutar Camion Table
6.  Ejecutar Camion DP
7.  Ejecutar Camion DP2
8.  Ejecutar Camion DP3
9.  Ejecutar Camion DPMD
10. Ejecutar Camion Greedy
11. Ejecutar pruebas de rendimiento
12. Salir
==========================================
Seleccione una opción:
4
Version recursiva
BENEFICIO TOTAL: 150.0
```

CamionTable

```
Seleccione una opción:
5
Version memoria
x   x   x   
```

CamionDP3



CamionDPMP

```
Seleccione una opción:
9
Version DPMP
MOCHILA DE CAPACIDAD (190)
Objeto 1        p=56,0        b=50,0        q=2
Objeto 2        p=59,0        b=50,0
Objeto 6        p=17,0        b=5,0
BENEFICIO TOTAL: 155.0
PESO TOTAL: 188
```

Greedy

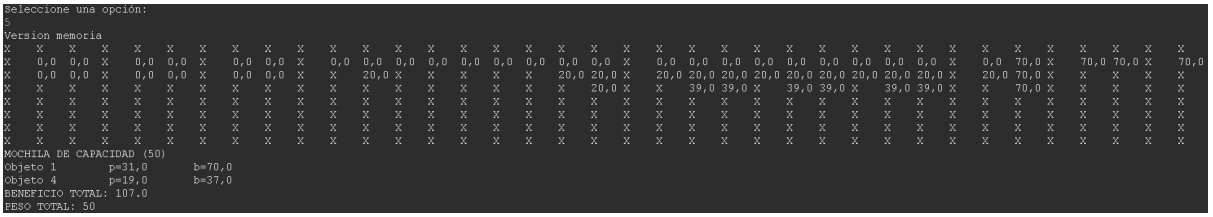
```
Seleccione una opción:
10
Version Greedy
MOCHILA DE CAPACIDAD (190)
Objeto 1        p=56,0        b=50,0
Objeto 2        p=59,0        b=50,0
Objeto 3        p=80,0        b=64,0        q=0.9375
BENEFICIO TOTAL: 160.0
PESO TOTAL: 190.0
```

P04

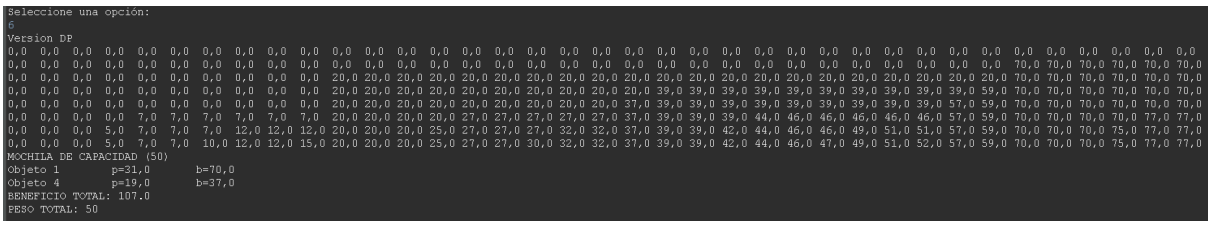
CamionRecursive

```
Cargando archivo "p04_c.txt"...
Archivo cargado correctamente
========================================
                MENU
========================================
1.  Cargar archivo
2.  Generar mochila aleatoria
3.  Crear mochila manual
4.  Ejecutar Camion Recursive
5.  Ejecutar Camion Table
6.  Ejecutar Camion DP
7.  Ejecutar Camion DP2
8.  Ejecutar Camion DP3
9.  Ejecutar Camion DPMD
10. Ejecutar Camion Greedy
11. Ejecutar pruebas de rendimiento
12. Salir
========================================
Seleccione una opción:
4
```

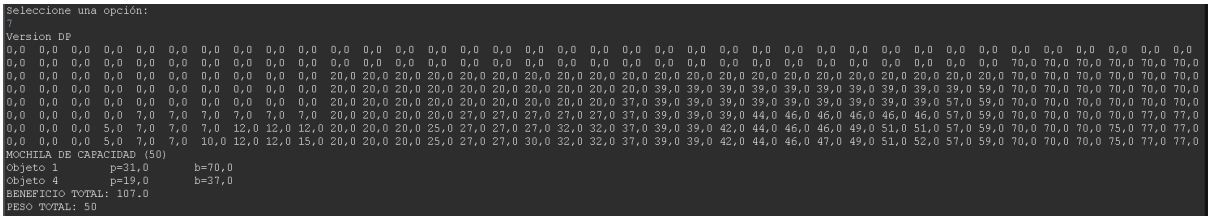
CamionTable



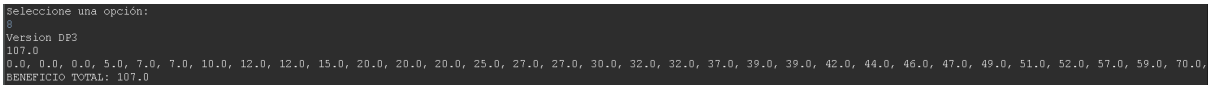
CamionDP



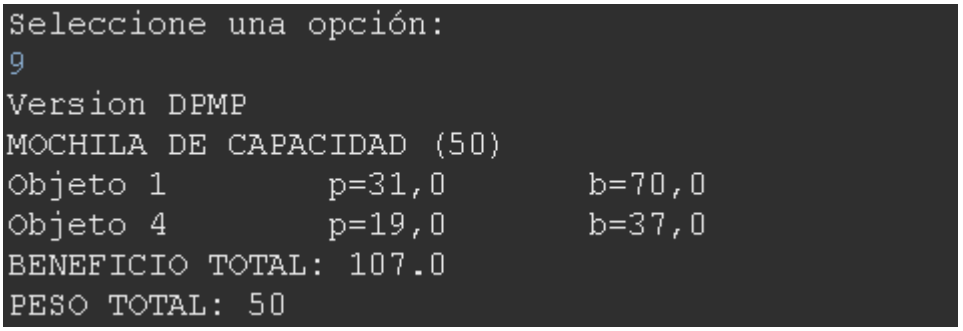
CamionDP2



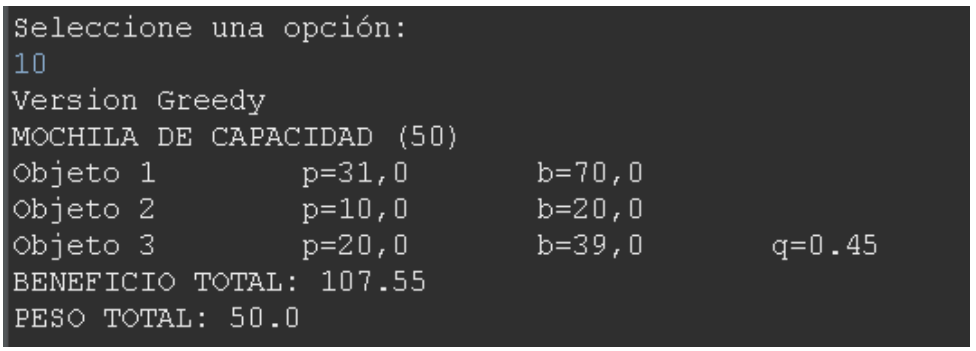
CamionDP3



CamionDPMP



Greedy



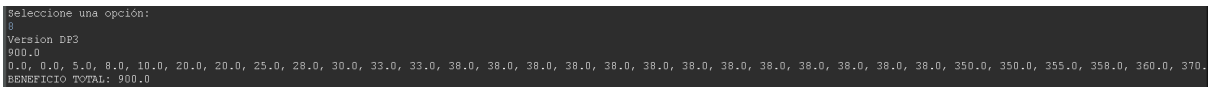
P05

CamionRecursive

```
Cargando archivo "p05_c.txt"...
Archivo cargado correctamente
==========================================
                  MENU
==========================================
1.  Cargar archivo
2.  Generar mochila aleatoria
3.  Crear mochila manual
4.  Ejecutar Camion Recursive
5.  Ejecutar Camion Table
6.  Ejecutar Camion DP
7.  Ejecutar Camion DP2
8.  Ejecutar Camion DP3
9.  Ejecutar Camion DPMD
10. Ejecutar Camion Greedy
11. Ejecutar pruebas de rendimiento
12. Salir
==========================================
Seleccione una opción:
4
Version recursiva
BENEFICIO TOTAL: 900.0
```

CamionTable

```
Seleccione una opción:
5
Version memoria
X   X   X   
```



CamionDPMP

```
Seleccione una opción:
9
Version DPMP
MOCHILA DE CAPACIDAD (104)
Objeto 1        p=25,0        b=350,0       q=4
Objeto 8        p=2,0         b=5,0         q=2
BENEFICIO TOTAL: 1410.0
PESO TOTAL: 104
```

Greedy

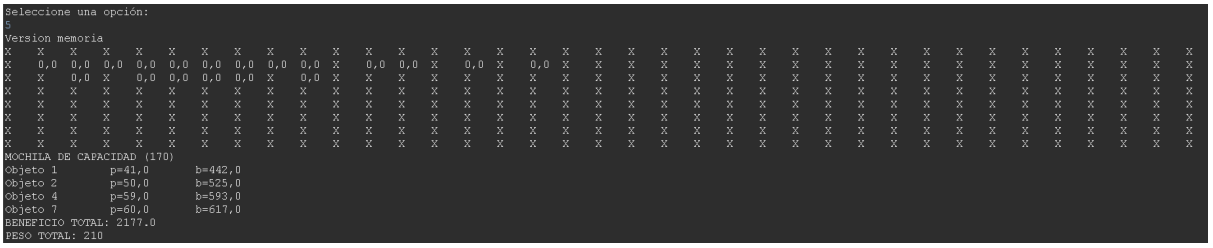
```
Seleccione una opción:
10
Version Greedy
MOCHILA DE CAPACIDAD (104)
Objeto 1        p=25,0       b=350,0
Objeto 2        p=35,0       b=400,0
Objeto 3        p=45,0       b=450,0      q=0.9777777777777777
BENEFICIO TOTAL: 1190.0
PESO TOTAL: 104.0
```

P06

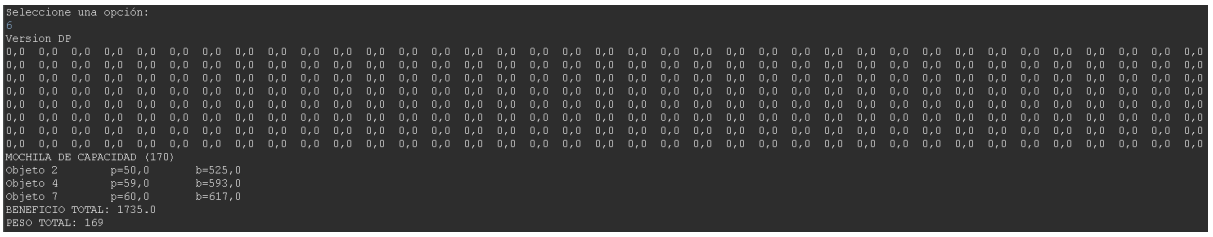
CamionRecursive

```
Cargando archivo "p06_c.txt"...
Archivo cargado correctamente
==========================================
                 MENU
==========================================
1.  Cargar archivo
2.  Generar mochila aleatoria
3.  Crear mochila manual
4.  Ejecutar Camion Recursive
5.  Ejecutar Camion Table
6.  Ejecutar Camion DP
7.  Ejecutar Camion DP2
8.  Ejecutar Camion DP3
9.  Ejecutar Camion DPMD
10. Ejecutar Camion Greedy
11. Ejecutar pruebas de rendimiento
12. Salir
==========================================
Seleccione una opción:
4
Version recursiva
BENEFICIO TOTAL: 1735.0
```

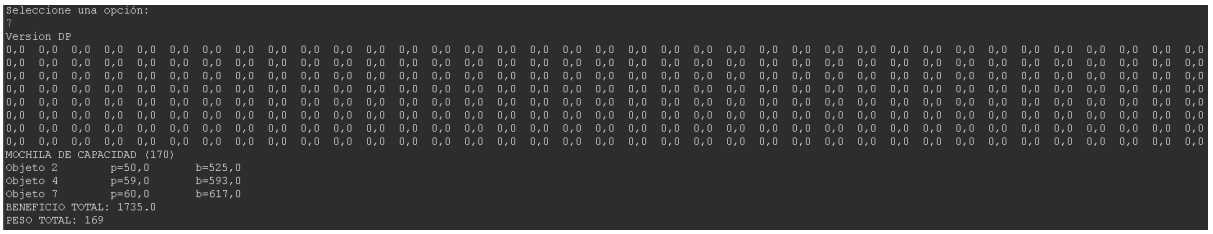
CamionTable



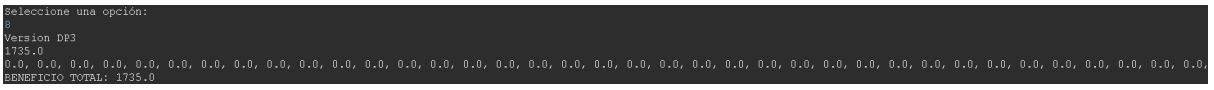
CamionDP



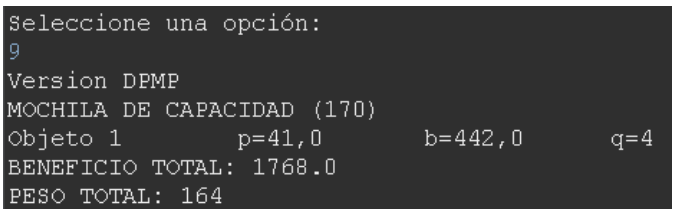
CamionDP2



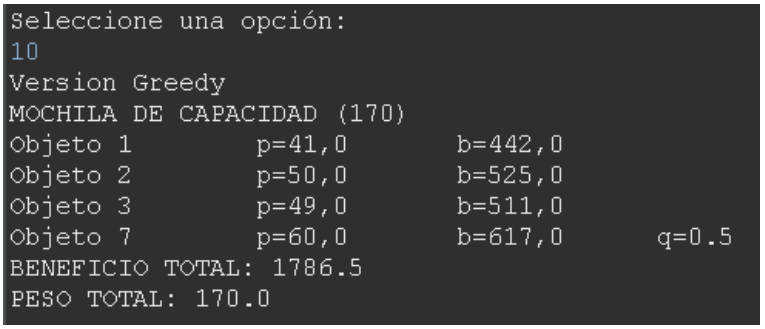
CamionDP3



CamionDPMP



Greedy



P07

CamionRecursive

```
Cargando archivo "p07_c.txt"...
Archivo cargado correctamente
==========================================
                  MENU
==========================================
1.  Cargar archivo
2.  Generar mochila aleatoria
3.  Crear mochila manual
4.  Ejecutar Camion Recursive
5.  Ejecutar Camion Table
6.  Ejecutar Camion DP
7.  Ejecutar Camion DP2
8.  Ejecutar Camion DP3
9.  Ejecutar Camion DPMD
10. Ejecutar Camion Greedy
11. Ejecutar pruebas de rendimiento
12. Salir
==========================================
Seleccione una opción:
4
Version recursiva
BENEFICIO TOTAL: 1458.0
```

CamionTable

```
Seleccione una opción:
5
Version memoria
X   X   X   
```

CamionDP

```
Seleccione una opción:
6
Version DP
0,0  0,0  0,0  
```

Greedy

```
Seleccione una opción:
10
Version Greedy
MOCHILA DE CAPACIDAD (750)
Objeto 1        p=70,0       b=135,0
Objeto 3        p=77,0       b=149,0
Objeto 7        p=90,0       b=173,0
Objeto 8        p=94,0       b=184,0
Objeto 9        p=98,0       b=192,0
Objeto 13       p=115,0      b=221,0      q=0.7217391304347827
Objeto 14       p=118,0      b=229,0
Objeto 15       p=120,0      b=240,0
BENEFICIO TOTAL: 1461.5043478260868
PESO TOTAL: 750.0
```

P08

CamionRecursive

```
Cargando archivo "p08_c.txt"...
Archivo cargado correctamente
==========================================
                 MENU
==========================================
1.  Cargar archivo
2.  Generar mochila aleatoria
3.  Crear mochila manual
4.  Ejecutar Camion Recursive
5.  Ejecutar Camion Table
6.  Ejecutar Camion DP
7.  Ejecutar Camion DP2
8.  Ejecutar Camion DP3
9.  Ejecutar Camion DPMD
10. Ejecutar Camion Greedy
11. Ejecutar pruebas de rendimiento
12. Salir
==========================================
Seleccione una opción:
4
Version recursiva
BENEFICIO TOTAL: 1.3549094E7
```

CamionDPMP

```
Seleccione una opción:
9
Version DPMP
MOCHILA DE CAPACIDAD (6404180)
Objeto 6        p=44328,0     b=97426,0     q=143
Objeto 20       p=31385,0     b=65731,0     q=2
BENEFICIO TOTAL: 1.406338E7
PESO TOTAL: 6401674
```

Greedy

```
Seleccione una opción:
10
Version Greedy
MOCHILA DE CAPACIDAD (6404180)
Objeto 1        p=382745,0    b=825594,0
Objeto 2        p=799601,0    b=1677009,0
Objeto 4        p=729069,0    b=1523970,0
Objeto 6        p=44328,0     b=97426,0
Objeto 9        p=823460,0    b=1679693,0   q=0.1397651373472907
Objeto 10       p=903959,0    b=1902996,0
Objeto 11       p=853665,0    b=1844992,0
Objeto 13       p=610856,0    b=1252836,0
Objeto 16       p=951111,0    b=2067538,0
Objeto 17       p=323046,0    b=675367,0
Objeto 20       p=31385,0     b=65731,0
Objeto 22       p=264724,0    b=577243,0
Objeto 23       p=224916,0    b=466257,0
Objeto 24       p=169684,0    b=369261,0
BENEFICIO TOTAL: 1.3580982522846282E7
PESO TOTAL: 6404180.0
```

Las imágenes que se han agregado es ejecutando todos los métodos con los archivos proporcionados en la práctica mostrando su solución, se han comprobado y la solución que se nos proporciona y la de nuestros métodos coinciden, pudiendo afirmar que los métodos funcionan correctamente. Para el archivo P08, hemos tenido un problema con los métodos camion Table, DP, DP2 y DP3, ya que al ejecutarlos se nos quedaba el eclipse congelado y nos obligaba a cerrar el programa o parar la ejecución sin proporcionarnos una salida.

BIBLIOGRAFÍA

https://people.sc.fsu.edu/~jburkardt/datasets/knapsack_01/knapsack_01.html
https://www.enjoyalgorithms.com/blog/zero-one-knapsack-problem